

一、名词解释

1. 地址映射：指将程序中的逻辑地址转换为内存中的物理地址的过程。它允许程序在编译或链接时使用虚拟地址，而无需关心程序实际运行时在内存中的位置。
2. 动态重定位：在程序执行期间，当访问指令或数据时，由硬件地址转换机构（如重定位寄存器）动态地将逻辑地址转换为物理地址。
3. 虚拟存储器：一种存储管理技术，通过使用外存作为内存的扩展，使得程序可以在比实际内存更大的空间上运行，而无需一次性全部调入内存。
4. 静态链接：在程序运行前，将各个模块连接成一个完整的可执行程序的过程。链接完成后，程序的结构不再改变。
5. 对换：在请求调页系统中，将暂时不用的进程的页调出到外存，以便将内存空间让给其他进程使用的一种技术。
6. 设备驱动程序：操作系统中负责控制和管理计算机硬件设备的程序，为应用程序提供与设备无关的接口。
7. SPOOLing：模拟脱机 I/O 操作的一种技术，将作业的输入数据先送到外存，再由外围处理机控制设备进行输入输出操作。
8. I/O 通道：一种特殊的处理器，专门用于控制 I/O 操作，能够执行 I/O 指令，具有一定的数据处理能力。
9. 文件系统：操作系统中用于存储和管理文件的软件和数据结构的集合，为用户提供更多的文件操作功能。
10. 目标文件：由编译器将源程序编译后生成的机器语言代码文件，包含了程序的机器指令和数据。
11. 文件的逻辑结构：指用户在使用文件时所看到的文件的组织形式，如顺序文件、索引文件等。
12. 有结构文件：文件中的数据具有一定的结构，如记录式文件、索引文件等，方便用户进行随机访问和数据处理。
13. 位示图：一种用于磁盘空间管理的技术，用一个位来表示磁盘的一个块是否被占用，位示图可以有效地管理磁盘空间的分配和回收，提高磁盘空间的利用率。
14. 程序接口：程序与其他程序、硬件或外部环境之间的交互接口，用于实现不同程序之间的通信和协作。
15. 系统调用：用户程序请求操作系统内核提供服务的一种方式，通过调用系统调用接口，应用程序可以请求操作系统执行如文件操作、进程管理等特权操作。
16. I/O 中断：当 I/O 设备完成一次输入输出操作或发生错误时，向 CPU 发出的中断信号，CPU 暂停当前任务，转而处理 I/O 中断请求。
17. 文件管理系统：操作系统中负责文件存储、读取、写入、删除等文件操作的子系统，为用户提供具体的文件操作接口。
18. 文件：由一组相关信息组成的集合，可以是程序、数据、文档等，是用户与操作系统交换信息的基本单位。
19. 文件的物理结构：指文件在存储设备上的物理存储方式，如连续结构、链接结构、索引结构等。
20. 文件的逻辑结构：指用户在使用文件时所看到的文件的组织形式，如顺序文件、索引文件等。

二、填空题

1. 绝对定位
2. 绝对装入 动态重定位装入 动态运行时装入

3. 静态链接 装入时动态链接 运行时动态链接
4. 空闲分区表 空闲分区链
5. 内存拼接
6. 固定 系统 可变 用户
7. 对性 离散性
8. 文件 目录 文件管理软件
9. 如何划分分区 如何分配内存 如何回收分区
10. 外部碎片
11. 分页 分段 段页式
12. 快表
13. 物理空间 逻辑空间
14. 模块独立性 便于段共享 便于动态链接 便于存储保护 用户可直接访问
15. 页表
16. 局部性 时间局部性 空间局部性。
17. 处理器 它具有自己的指令和程序计数器 它具有自己的 I/O 指令集
18. 记录式文件 索引文件 直接存取文件
19. 单级索引 多级索引 索引分配
20. 动态重定位
21. 用户层 设备无关层 设备驱动层
22. 顺序文件 索引文件 直接存取文件 直接存取文件
23. 高速缓存 主存 辅存
24. 动态分区分配 静态分区分配
25. 存储设备 文件的使用方式
26. 顺序文件 索引文件 随机文件
27. 连续分配 链接分配 索引分配
28. 相对路径 绝对路径
29. 隐式链接 显式链接
30. 直接共享 间接共享
31. 块设备 字符设备 独占设备 共享设备 虚拟设备
32. DMA 方式 通道方式
33. 单缓冲 循环缓冲 缓冲池
34. 寻道时间 传输时间

三、简答题

1. 计算机存储器系统主要有以下层次：

高速缓存（Cache）：位于 CPU 和主存之间，用于缓存频繁访问的数据和指令，以提高 CPU 访问速度。其特点是访问速度快，容量小，价格高。

主存（Memory）：用于存储当前正在运行的程序和数据，具有较高的访问速度和较大的容量，但价格相对较高。

辅存（Secondary Memory）：如硬盘、光盘等，用于长期存储数据和程序，具有大容量、价格低的特点，但访问速度较慢。

从存储性能角度分析，存储系统需要在访问速度、存储容量和成本之间进行权衡。高速缓存提供了快速的数据访问，但容量有限；主存具有较高的访问速度和适中的容量，能满足大多数程序的运行需求；辅存则提供了大容量的长期存储，但访问速度较慢。为了提高存储系统

的整体性能，通常采用多层次的存储体系结构，将不同层次的存储设备结合使用，通过缓存技术和虚拟存储技术来实现高性能和大容量的平衡。

2. 计算机程序从代码编写到运行完毕的主要过程如下：

编写代码：程序员使用高级语言（如 C、Java 等）编写源代码。

预处理：对源代码中的预处理指令进行处理，如宏定义、文件包含等。

编译：将预处理后的源代码编译成汇编代码或目标代码。编译器会对代码进行语法检查和语义分析，确保代码的正确性。

汇编：将汇编代码转换成机器语言的目标代码，生成目标文件。

链接：将多个目标文件链接成一个完整的可执行文件，解决模块间的外部引用问题。

加载：将可执行文件加载到内存中，分配必要的内存空间和资源。

运行：操作系统调度 CPU 执行加载到内存中的程序，程序开始运行，与用户或其他程序进行交互。

退出：程序运行结束后，释放所占用的资源，返回控制权给操作系统。

3. 程序链接的主要方式包括：

静态链接：在程序运行前，将各个模块连接成一个完整的可执行程序。链接完成后，程序的结构不再改变。其优点是程序运行时不需要额外的链接操作，执行效率高；缺点是生成的可执行文件较大，且不同程序之间无法共享代码模块。

装入时动态链接：在程序装入内存时进行链接，将程序中引用的动态链接库（DLL）等模块加载到内存并链接到程序中。其优点是可以共享代码模块，减少内存占用；缺点是程序装入时间可能较长，因为需要进行链接操作。

运行时动态链接：在程序运行时，当需要调用某个函数或模块时才进行链接。其优点是进一步提高了代码共享程度，减少了内存占用；缺点是增加了程序运行时的开销，链接过程可能会导致程序运行速度变慢。

它们的共同点是都将不同的模块组合成一个完整的可执行程序；不同点在于链接的时间和方式不同，静态链接在编译时完成，生成独立的可执行文件；装入时动态链接在装入内存时完成，共享代码模块；运行时动态链接在程序运行时按需链接，灵活性更高但运行时开销较大。

4. 内存动态分区分配算法根据搜索空闲区的方式可分为以下类型：

首次适应算法（First Fit）：从内存的开始处查找第一个足够大的空闲分区来满足内存分配请求。该算法倾向于在内存的低地址部分分配分区，容易在内存的高地址部分留下较大的空闲分区，但可能会导致内存碎片的分布不均匀。

循环首次适应算法（Next Fit）：从上次查找结束的位置开始继续查找下一个足够大的空闲分区。该算法试图减少首次适应算法中对低地址区域的频繁搜索，但可能会导致内存分配的不均衡，某些区域可能被过度使用而其他区域则空闲。

最佳适应算法（Best Fit）：在整个空闲分区列表中查找最适合请求大小的空闲分区，即找到比请求大小稍大的最小空闲分区进行分配。该算法倾向于产生较小的内存碎片，但可能会增加分配过程中的搜索时间，且可能导致剩下的空闲分区过于细碎，难以满足后续较大的内存请求。

最坏适应算法（Worst Fit）：在整个空闲分区列表中查找最大的空闲分区进行分配，目的是留下较大的空闲分区，减少内存碎片。该算法可能会导致大空闲分区被分割成较小的分区，可能会影响后续的内存分配效率。

这些算法的核心思想都是在空闲分区中找到合适的分区来满足内存分配请求，但它们的搜索策略和目标不同，导致在内存分配效率、碎片控制和搜索时间等方面各有优劣。

5. 典型的内存动态分区算法包括首次适应算法、循环首次适应算法、最佳适应算法和最坏适应算法。典型的 CPU 调度算法包括先来先服务（FCFS）、短作业优先（SJF）、优先级调

度、时间片轮转（RR）等。

相同点：内存动态分区算法和 CPU 调度算法都是操作系统中用于资源分配的策略，旨在合理分配有限的资源（内存或 CPU），提高系统的性能和利用率。

不同点：内存动态分区算法关注的是如何在内存中分配和管理分区，以满足进程的内存需求，主要考虑因素包括内存碎片、分配速度和内存利用率等；而 CPU 调度算法关注的是如何合理安排进程在 CPU 上的执行顺序，主要考虑因素包括进程的等待时间、周转时间、响应时间以及系统的吞吐量等。两者所针对的资源类型不同，优化目标也不同。

6.典型的页面置换算法包括先进先出（FIFO）、最近最少使用（LRU）、时钟算法等。典型的 CPU 调度算法包括先来先服务（FCFS）、短作业优先（SJF）、优先级调度、时间片轮转（RR）等。

相同点：页面置换算法和 CPU 调度算法都是在资源有限的情况下，选择合适的策略来决定资源的使用顺序，以提高系统的性能。它们都需要考虑到资源的使用效率和公平性等问题。

不同点：页面置换算法主要用于虚拟存储器系统中，当内存不足时决定将哪个页面调出内存，以腾出空间给新的页面。其主要目标是减少缺页中断次数，提高内存的利用率。而 CPU 调度算法则是决定哪个进程先占用 CPU 运行，主要目标是使 CPU 尽可能保持忙碌状态，同时满足进程的响应时间、周转时间等性能指标。两者所针对的资源不同，一个是内存资源，一个是处理器资源，其优化目标和影响因素也存在差异。

7.典型的页面置换算法包括先进先出（FIFO）、最近最少使用（LRU）、时钟算法等。典型的磁盘调度算法包括先来先服务（FCFS）、最短寻道时间优先（SSTF）、扫描算法（SCAN）、循环扫描算法（C-SCAN）等。

相同点：页面置换算法和磁盘调度算法都是用于优化资源分配和访问效率的算法。它们都面临着如何在多个请求之间做出选择的问题，需要考虑公平性、效率和性能等多方面的因素。

不同点：页面置换算法主要应用于虚拟存储系统中的内存管理，目的是减少缺页中断次数，提高内存的利用率，其决策主要基于页面的使用情况和内存的可用空间等内存相关的因素。而磁盘调度算法则是针对磁盘 I/O 操作的调度，目的是减少磁盘的寻道时间和旋转延迟，提高磁盘的 I/O 性能，其决策主要基于磁盘的物理位置（如磁道、扇区等）以及 I/O 请求的先后顺序等因素。两者所针对的资源类型不同，一个是内存资源，一个是磁盘设备，其优化目标和依据的因素也有所不同。

8. 分页存储的地址变换过程：分页存储系统中，逻辑地址被分为页号和页内偏移量。当进程访问一个逻辑地址时，系统通过页表将页号转换成对应的物理块号，然后与页内偏移量组合成物理地址。页表的基址通常存放在页表寄存器中，以便快速访问页表。

分段存储的地址变换过程：在分段存储系统中，逻辑地址由段号和段内偏移量组成。地址变换时，系统根据段号查找段表，获取该段在内存中的起始地址（段基址），然后将段基址与段内偏移量相加得到物理地址。段表的起始地址一般存放在段表寄存器中。

段页存储的地址变换过程：段页存储系统将分段和分页相结合。首先，逻辑地址被分为段号、页号和页内偏移量。系统先根据段号查找段表，找到对应的页表起始地址；然后根据页号查找该页表，获取物理块号；最后将物理块号与页内偏移量组合成物理地址。这种地址变换过程需要两次查表操作（段表和页表），增加了地址变换的复杂度，但同时也结合了分段和分页的优点。

9. 中断处理程序通常包括以下步骤：

保存现场：当中断发生时，CPU 暂停当前正在执行的程序，将程序计数器、寄存器等关键信息保存到系统栈或其他指定的存储区域，以确保在中断处理完成后能够正确恢复被打断的程序的执行状态。

分析中断源：中断处理程序需要确定是哪个中断源发出的中断请求，以便采取相应的处理措

施。这可以通过检查中断控制器的状态寄存器或其他相关硬件设施来实现。

对中断进行处理：根据中断源的类型和性质，执行相应的中断处理任务。例如，对于外部设备的 I/O 中断，可能需要读取或写入数据；对于时钟中断，可能需要更新系统时钟或处理定时任务等。

恢复现场：中断处理完成后，将之前保存的程序计数器、寄存器等信息恢复到 CPU 的相应位置，使被中断的程序能够从断点处继续执行。

返回被中断程序：中断处理程序执行完所有任务后，通过执行中断返回指令，将控制权交还给被中断的程序，使其继续运行。

10. 按用途分类：

系统文件：用于存储操作系统内核、驱动程序、系统配置文件等与操作系统运行密切相关的数据，是操作系统正常运行的基础。

用户文件：由用户创建和使用的文件，包含用户的数据、文档、程序代码等，用于满足用户的特定需求和应用。

库文件：包含了预编译的函数、模块和资源，可供多个程序共享使用，方便程序的开发和构建，提高代码的复用性和开发效率。

按数据类型分类：

文本文件（ASCII 文件）：文件中的数据以文本形式存储，由字符序列组成，可以用文本编辑器查看和编辑，如 .txt 文件。

二进制文件：文件中的数据以二进制格式存储，通常包含程序的机器代码、图像、音频、视频等多媒体文件以及一些特定格式的数据文件，不能直接用文本编辑器正确显示和编辑。

按组织和管理方式分类：

普通文件：由用户或程序创建的常规文件，用于存储数据、文档、程序代码等，具有特定的文件名和扩展名，按照文件系统的规定进行组织和管理。

目录文件：用于组织和管理其他文件的特殊文件，包含文件名和对应的文件系统的位置信息（如 inode 号等），构成文件系统的层次结构，方便用户查找和管理文件。

11. 文件目录的分类及相应查询方式如下：

单级目录结构：系统中只建立一个目录，所有文件都登记在这个目录中。查询时，直接在该目录中进行顺序搜索或采用其他索引方式快速定位文件。

两级目录结构：分为主文件目录（MFD）和用户文件目录（UFD）。主文件目录用于登记所有用户的 UFD，每个用户拥有一个 UFD，用于存放该用户的所有文件。查询时，先在主文件目录中找到对应的 UFD，再在 UFD 中查找目标文件。

树型目录结构：将文件系统组织成树形结构，每个目录可以包含子目录和文件。查询时，可以从根目录开始，按照路径名逐级查找目标文件或目录。根据路径的起点不同，可分为绝对路径（从根目录开始）和相对路径（从当前工作目录开始）两种查询方式。

无环图目录结构：允许用户在不同目录之间建立文件的链接，形成无环图结构。查询时，可以通过不同的路径访问同一个文件，增加了文件访问的灵活性。

通用目录结构：在无环图目录结构的基础上，允许建立目录的链接，进一步增强了目录结构的灵活性和复杂性。查询方式与无环图目录结构类似，可以根据路径进行灵活的文件查找。

12. 文件结构的三种主要组织方式如下：

顺序文件组织方式：文件中的记录按照一定的顺序（如按关键字大小顺序）依次存储。其优点是访问效率高，特别是在进行顺序访问时可以快速读取文件中的所有记录；缺点是插入和删除操作较为困难，可能需要移动大量记录，导致操作效率低下。

索引文件组织方式：在文件中建立索引结构，通过索引可以快速定位到文件中的特定记录。其优点是能够提供快速的随机访问能力，方便对指定记录的查找、插入和删除操作；缺点是

增加了索引的存储开销，且索引的维护需要一定的处理时间，可能会降低文件的整体性能。
直接存取文件（哈希文件）组织方式：根据记录的关键字通过哈希函数直接计算出该记录在文件中的存储位置，实现快速的随机访问。其优点是能够提供极快的随机访问速度，插入和删除操作也相对高效；缺点是哈希冲突可能会导致存储空间的浪费，且当文件规模较大时，哈希函数的设计和优化较为复杂。

对比分析：

顺序文件组织方式在数据顺序访问时效率最高，适合于对数据进行批量处理的应用场景，但对于需要频繁进行随机访问、插入和删除操作的情况则不太适用。

索引文件组织方式在随机访问和数据更新方面具有较好的性能，能够满足大多数应用程序对文件操作的需求，但需要额外的存储空间来保存索引，并且索引的维护可能会带来一定的性能开销。

直接存取文件组织方式在随机访问速度上具有显著优势，能够快速定位和访问指定记录，但在处理哈希冲突和存储空间利用率方面可能存在一定的问题，适用于对随机访问性能要求较高的应用场景。

13. 内存的存储分配方式主要包括：

连续分配方式：将内存中的一块连续的空间分配给一个进程。连续分配方式又可分为单用户连续分配、固定分区分配和动态分区分配等。其优点是管理简单，地址变换方便；缺点是容易产生内存碎片，降低了内存的利用率。

离散分配方式：将内存分割成多个离散的块，进程可以占用多个不连续的内存块。常见的离散分配方式有分页存储管理和分段存储管理等。其优点是可以有效地利用内存空间，减少内存碎片；缺点是地址变换过程相对复杂，需要硬件支持。

外存的存储分配方式主要包括：

连续分配方式：为一个文件分配连续的磁盘块。其优点是文件的读写速度快，适合于顺序访问的文件；缺点是难以找到足够大的连续空间来分配文件，容易产生外部碎片，且文件的扩展和缩减较为困难。

链接分配方式：文件由一系列离散的磁盘块组成，每个磁盘块中包含指向下一个磁盘块的指针。其优点是可以充分利用磁盘空间，不存在外部碎片问题，文件的扩展和缩减比较方便；缺点是文件的访问速度较慢，因为需要通过指针链进行顺序访问，不适合随机访问。

索引分配方式：为每个文件建立一个索引表，索引表中包含了文件所有磁盘块的地址。其优点是既能高效地利用磁盘空间，又能方便地支持随机访问，文件的扩展和管理也比较方便；缺点是需要额外的存储空间来保存索引表，并且索引表的维护需要一定的开销。

内存和外存存储分配方式的异同点：

相同点：内存和外存的存储分配方式都面临着如何有效地利用存储空间、提高存储资源利用率的问题，在分配和回收存储空间时都需要考虑如何满足用户的需求并优化系统的性能。

不同点：内存主要用于临时存储正在运行的程序和数据，对访问速度要求极高，因此内存的分配方式更注重如何快速地分配和回收内存空间，减少内存碎片对系统性能的影响。而外存用于长期存储数据，对访问速度的要求相对较低，更侧重于如何高效地管理大容量的存储空间，支持文件的顺序访问和随机访问等多种访问方式，并且需要考虑文件的扩展性、安全性和可靠性等因素。

14. 提高磁盘 I/O 速度的途径主要有以下几种：

优化磁盘调度算法：采用合适的磁盘调度算法，如最短寻道时间优先（SSTF）、扫描算法（SCAN）、循环扫描算法（C-SCAN）等，减少磁盘的平均寻道时间和旋转延迟，提高磁盘的 I/O 性能。

增加磁盘缓存：在磁盘控制器或主机内存中设置缓存，用于暂存频繁访问的数据。当 CPU 访

访问磁盘数据时，如果数据在缓存中，则可以直接从缓存中读取，减少对磁盘的访问次数，提高 I/O 速度。

采用高速磁盘：使用转速更快、寻道时间更短的磁盘，如固态硬盘（SSD），可以显著提高磁盘的读写速度。SSD 采用闪存芯片作为存储介质，没有机械部件，具有读写速度快、功耗低、可靠性高等优点。

磁盘阵列（RAID）技术：通过将多个磁盘组合成一个虚拟的磁盘阵列，利用数据条带化、镜像、奇偶校验等技术，提高磁盘的 I/O 性能、可靠性和可用性。例如，RAID 0 通过数据条带化将数据分散存储在多个磁盘上，能够提高读写速度；RAID 1 通过镜像技术将数据同时写入两个磁盘，提高数据的可靠性，并在一定程度上也能够提高读取速度。

优化文件系统：选择合适的文件系统类型，并对文件系统进行优化配置，如调整磁盘块大小、优化文件索引结构、定期进行磁盘整理等，以减少文件的碎片，提高文件的读写效率，进而提高磁盘 I/O 速度。

重叠操作：让 CPU 的操作和磁盘的 I/O 操作重叠进行，使 CPU 在等待磁盘 I/O 完成期间能够执行其他任务，充分利用 CPU 和磁盘的并行处理能力，提高系统的整体性能。

提高数据传输率：采用高速的接口总线，如 SATA、SCSI、USB 3.0 等，增加磁盘与主机之间的数据传输带宽，减少数据传输时间，从而提高磁盘 I/O 速度。